Never paste a JWT or key from a production environment into a website.

6.2.4 Validating a signed JWT

To validate a JWT you first parse the JWS Compact Serialization format and then use the `JWSVerifier` object to verify the signature. The Nimbus `MACVerifier` will calculate the correct HMAC tag and then compare it to the tag attached to the JWT using a constant-time equality comparison, just like you did in the `HmacTokenVerifier`. The Nimbus library also takes care of basic security checks, such as making sure that the algorithm header is compatible with the verifier (preventing the algorithm mix up attacks discussed in section 6.2) and that there are no unrecognized critical headers. After the signature has been verified, you can extract the JWT claims set and verify any constraints. In this case, you just need to check that the expected audience value appears in the audience claim, and then set the token expiry from the JWT expiry time claim. The `TokenController` will ensure that the token hasn't expired. Listing 6.4 shows the full JWT validation logic. Open the SignedJwtTokenStore.java file and replace the `read()` method with the contents of the listing.

Listing 6.4 Validating a signed JWT

```
    @Override
    public Optional<Token> read(Request request, String
tokenId) {
        try {
            var jwt = SignedJWT.parse(tokenId);    #A
```